🚆 Guide Universel d'Installation & Déploiement — CAMRAIL RailMind Lite

> **Statut : Document de référence unique pour l'installation intégrale**
> **Cible : Environnements Linux (Ubuntu/Debian) et Windows (Server/Desktop)**
> **Architecture : 100 % Souverain, Local & Déconnecté (Air-Gapped)**

📑 Sommaire Général

1. Principes d'Architecture & Conventions de Chemins
2. Inventaire Exhaustif des Poids, Modèles IA et Binaires Tiers
3. Étape 1 : Prérequis Système Fondamentaux
   - A. Python 3.12+
   - B. Node.js 20+ / 22+ LTS
   - C. PostgreSQL 16+ et pgvector
   - D. Utilitaires OCR & PDF : Poppler et Tesseract
4. Étape 2 : Moteur LLM Local — Ollama & Téléchargement des Poids
5. Étape 3 : Modèle d'Embedding Sémantique — Téléchargement & Emplacement
6. Étape 4 : Configuration des Variables d'Environnement (.env)
   - Les 3 Profils Matériels Prédéfinis (A, B, C)
   - Dictionnaire des Variables Critiques
7. Étape 5 : Initialisation de la Base de Données (Alembic & Seed)
8. Étape 6 : Démarrage du Backend FastAPI
9. Étape 7 : Installation & Démarrage du Frontend Next.js
10. Étape 8 : Déploiement Conteneurisé Clé en Main (Docker Compose)
11. Procédure de Transfert 100 % Hors-Ligne (Serveur Air-Gapped)
12. Guide de Dépannage & Erreurs Fréquentes

---

1. Principes d'Architecture & Conventions de Chemins

A. Souveraineté et Étancheité Réseau

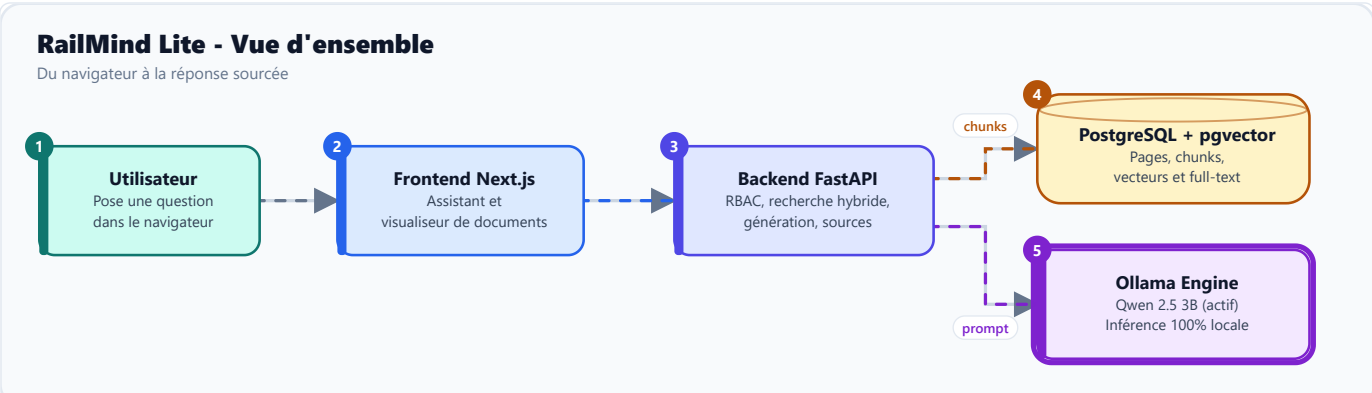
RailMind Lite est conçu pour respecter les contraintes strictes de confidentialité de **CAMRAIL** :

- **Zéro appel cloud externe** : Aucun flux réseau sortant vers OpenAI, Google, Anthropic ou Hugging Face en production.
- **Inférence 100 % locale** : Génération textuelle opérée par **Ollama** et vectorisation sémantique opérée par **SentenceTransformers** depuis des fichiers locaux.
- **Circuit fermé (Air-Gapped)** : Le système fonctionne sans connexion Internet active une fois les modèles et dépendances installés.

B. Conventions de Chemins d'Accès

Les chemins d'installation varient d'une machine à l'autre (utilisateurs différents, lettres de disques `C:`, `D:`, arborescences `/home/` ou `/opt/`).
Dans ce guide :

- `<racine_projet>` désigne le dossier racine où le code source est déposé.
  - *Exemple Linux :* `/opt/camrail/railmind` ou `~/camrail-main`
  - *Exemple Windows :* `D:\camrail-main` ou `C:\Projets\camrail`
- Chaque section présente les commandes **Linux (Bash)** et **Windows (PowerShell / CMD)** côte-à-côte.



2. Inventaire Exhaustif des Poids, Modèles IA et Binaires Tiers

Voici le tableau de synthèse des éléments externes à installer, leur raison d'être, leur taille et leur emplacement précis :

COMPOSANT	RÔLE DANS RAILMIND LITE	TAILLE	EMPLACEMENT CIBLE (LINUX)	EMPLACEMENT CIBLE (WINDOWS)	COMMANDE DE TEST / VALIDATION
Ollama (Binaire)	Moteur d'inférence LLM local exécutant les modèles de langage	~100 Mo	`/usr/local/bin/ollama`	`C:\Users\<user>\AppData\Local\Programs\Ollama`	`ollama --version`
Poids LLM `qwen2.5:3b`	**Modèle de référence actif.** Recommandé pour CAMRAIL : haute fidélité au français, respect strict des consignes et acronymes	~1.9 Go	`/usr/share/ollama/.ollama/models/` ou `~/.ollama/models/`	`%USERPROFILE%\.ollama\models\`	`ollama list`
Poids LLM `llama3.2:3b`	Modèle alternatif léger pour CPU modeste	~2.0 Go	`/usr/share/ollama/.ollama/models/` ou `~/.ollama/models/`	`%USERPROFILE%\.ollama\models\`	`ollama list`
Modèle d'Embedding `multilingual-e5-small`	Vectorisation des documents et requêtes (384 dimensions)	~450 Mo	`<racine_projet>/backend/models/multilingual-e5-small/`	`<racine_projet>\backend\models\multilingual-e5-small\`	Présence des fichiers `model.safetensors` et `tokenizer.json`
Poppler (`poppler-utils` **)**	Découpage des PDF en pages individuelles et génération de vignettes	~30 Mo	`/usr/bin/pdftoppm`	Dossier choisi + ajouté au `PATH` (ex: `C:\Program Files\poppler\Library\bin`)	`pdftoppm -v`
Tesseract OCR	Reconnaissance optique de texte sur les scans ou documents numérisés	~50 Mo	`/usr/bin/tesseract`	`C:\Program Files\Tesseract-OCR\tesseract.exe`	`tesseract --version`
Fichier `fra.traineddata`	Dictionnaire linguistique français pour l'OCR Tesseract	~15 Mo	`/usr/share/tesseract-ocr/5/tessdata/fra.traineddata`	`C:\Program Files\Tesseract-OCR\tessdata\fra.traineddata`	`tesseract --list-langs` (doit afficher `fra`)
Extension `pgvector`	Permet à PostgreSQL d'indexer et rechercher des vecteurs (index HNSW)	~5 Mo	Extension installée dans le cluster PostgreSQL	Extension compilée dans PostgreSQL	Requête SQL : `CREATE EXTENSION IF NOT EXISTS vector;`

3. Étape 1 : Prérequis Système Fondamentaux

A. Python 3.12+

Linux (Ubuntu / Debian) :

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv python3-dev
python3 --version  # Doit être >= 3.12
```

Windows :

1. Téléchargez l'installateur officiel Python 3.12+ x64 depuis python.org.
2. **Important :** Cochez impérativement la case **"Add python.exe to PATH"** lors de l'installation.
3. Vérifiez dans PowerShell :

```
python --version
```

B. Node.js 20+ / 22+ LTS

Linux :

```
# Installation via NodeSource (version 22 LTS recommandée)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
node -v
npm -v
```

Windows :

```
# Option 1 : Via Winget
winget install OpenJS.NodeJS.22 --accept-package-agreements --accept-source-agreements

# Option 2 : Téléchargement manuel du .msi sur nodejs.org
# Vérification dans PowerShell :
node -v
npm -v
```

C. PostgreSQL 16+ et pgvector

RailMind Lite utilise PostgreSQL pour le stockage transactionnel et l'extension `pgvector` pour l'indexation sémantique HNSW des chunks de documents.

Linux :

```
sudo apt install -y postgresql postgresql-contrib
# Installation de pgvector (paquet officiel PostgreSQL)
sudo apt install -y postgresql-16-pgvector

# Démarrage du service
sudo systemctl enable --now postgresql

# Création sécurisée de la base, de l'extension et du rôle applicatif restreint :
# Vous pouvez exécuter directement le script durci fourni :
sudo -u postgres psql -f deploy/postgres/init.sql

# Ou manuellement en ligne de commande :
sudo -u postgres psql -c "CREATE EXTENSION IF NOT EXISTS vector;"
sudo -u postgres psql -c "CREATE ROLE railmind_app WITH LOGIN PASSWORD 'railmind_app_pass' NOSUPERUSER NOCREATEDB NOCREATEROLE;"
sudo -u postgres psql -c "ALTER DATABASE railmind OWNER TO railmind_app;"
sudo -u postgres psql -d railmind -c "ALTER SCHEMA public OWNER TO railmind_app;"
```

Windows :

1. Installez PostgreSQL 16+ via l'installateur officiel EDB.
2. Téléchargez la release de `pgvector` pour Windows correspondant à votre version de PostgreSQL (ex: release GitHub `pgvector/pgvector`).
3. Copiez :
   - `vector.dll` dans `C:\Program Files\PostgreSQL\16\lib\`
   - `vector.control` et `vector--*.sql` dans `C:\Program Files\PostgreSQL\16\share\extension\`
4. Dans `psql` (ou pgAdmin) connecté en super-utilisateur `postgres` :

```
CREATE DATABASE railmind;
\c railmind;
-- Exécution du script de durcissement (ou contenu de deploy/postgres/init.sql) :
CREATE EXTENSION IF NOT EXISTS vector;
CREATE ROLE railmind_app WITH LOGIN PASSWORD 'railmind_app_pass' NOSUPERUSER NOCREATEDB NOCREATEROLE;
ALTER DATABASE railmind OWNER TO railmind_app;
ALTER SCHEMA public OWNER TO railmind_app;
GRANT ALL PRIVILEGES ON DATABASE railmind TO railmind_app;
GRANT ALL ON SCHEMA public TO railmind_app;
```

D. Utilitaires OCR & PDF : Poppler et Tesseract

🐧 **Linux :**

```
# Poppler pour la découpe de documents PDF
sudo apt install -y poppler-utils

# Tesseract OCR et son modèle linguistique français
sudo apt install -y tesseract-ocr tesseract-ocr-fra

# Vérification :
pdftoppm -v
tesseract --list-langs  # Doit lister 'fra' et 'eng'
```

🪟 **Windows :**

1. **Poppler :**
   - Téléchargez l'archive binaire `poppler-windows` (depuis GitHub releases `oschwartz10612/poppler-windows`).
   - Décompressez-la dans un dossier fixe, par exemple : `C:\Program Files\poppler`.
   - Ajoutez le dossier `C:\Program Files\poppler\Library\bin` à votre variable d'environnement système `PATH`.
   - Testez dans un **nouveau** terminal PowerShell : `pdftoppm -v`.
2. **Tesseract OCR :**
   - Téléchargez l'installateur Windows `tesseract-ocr-w64-setup` (UB-Mannheim).
   - Lors de l'installation, cochez le composant de données linguistiques additionnelles : **French (`fra.traineddata`)**.
   - Si le modèle français n'a pas été coché, téléchargez `fra.traineddata` depuis le dépôt GitHub `tesseract-ocr/tessdata` et déposez-le dans : `C:\Program Files\Tesseract-OCR\tessdata\fra.traineddata`
   - Ajoutez `C:\Program Files\Tesseract-OCR` au `PATH`, ou déclarez son chemin dans le fichier `.env` :

     ```
     TESSERACT_CMD_PATH=C:\Program Files\Tesseract-OCR\tesseract.exe
     ```

---

4. Étape 2 : Moteur LLM Local — Ollama & Téléchargement des Poids

Pourquoi installer Ollama et ses poids ?

Ollama est le moteur d'inférence qui exécute localement le Grand Modèle de Langage (LLM). Il lit les fragments de documents retrouvés par la recherche sémantique et rédige la réponse structurée en français pour l'agent CAMRAIL, **sans jamais envoyer de données à l'extérieur**.

A. Installation d'Ollama

🐧 **Linux :**

```
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl enable --now ollama
```

🪟 **Windows :**

Téléchargez et exécutez l'installateur officiel : https://ollama.com/download/windows.
Ollama démarre automatiquement en tâche de fond (icône dans la barre des tâches).

B. Téléchargement du Modèle LLM

Exécutez dans votre terminal selon votre configuration :

```
# 1. Modèle standard de référence (RECOMMANDÉ pour CAMRAIL) :
# Supériorité en français technique, respect des acronymes (VACMA, DVL, AVL) et citations fiables
ollama pull qwen2.5:3b

# 2. Modèle alternatif standard (Meta) :
ollama pull llama3.2:3b

# 3. Modèle étendu si vous disposez d'un serveur dédié avec GPU (Profil C) :
ollama pull llama3.1:8b
```

> 💡 **Pourquoi Qwen 2.5 3B est le modèle recommandé ?**
> Les tests comparatifs menés sur le corpus ferroviaire de CAMRAIL ont démontré que `qwen2.5:3b` applique avec une rigueur supérieure les consignes strictes du prompt système (pas de phrases creuses du type "*Selon les extraits...*", respect sans substitution des codes réglementaires et des registres d'exploitation, abstention factuelle quand l'information n'est pas trouvée).

C. Où sont stockés les poids téléchargés ?

Ollama stocke les tenseurs binaires dans un dossier système :

- **Linux :** `/usr/share/ollama/.ollama/models` (ou `~/.ollama/models`)
- **Windows :** `%USERPROFILE%\.ollama\models` (généralement `C:\Users\<VotreUtilisateur>\.ollama\models`)

D. Vérification :

```
ollama list
# Doit afficher :
# NAME            ID              SIZE      MODIFIED
# qwen2.5:3b      357c53fb659c    1.9 GB    ...
# llama3.2:3b     478be6f61dbe    2.0 GB    ...
```

---

5. Étape 3 : Modèle d'Embedding Sémantique — Téléchargement & Emplacement

Pourquoi installer le modèle d'embedding ?

Le modèle d'embedding (SentenceTransformer) convertit chaque paragraphe documentaire et chaque question en un vecteur mathématique de 384 dimensions. C'est ce qui permet au moteur de retrouver instantanément la bonne procédure même si l'utilisateur formule sa question avec des mots différents.

A. Emplacement Cible

Le dossier cible est défini par la variable `MODELS_DIR` dans le fichier `.env` :

```
<racine_projet>/backend/models/multilingual-e5-small/
```

Ce dossier doit contenir les fichiers suivants :

- `model.safetensors` (ou `pytorch_model.bin`) — Poids des neurones
- `tokenizer.json`, `tokenizer_config.json`, `vocab.txt` — Vocabulaire multilingue
- `config.json`, `modules.json` — Configuration d'architecture

B. Téléchargement Automatisé via le script fourni

Depuis la racine du projet, exécutez :

Linux :

```
cd <racine_projet>/backend
source venv/bin/activate
python scripts/prepare_models.py
```

Windows (PowerShell) :

```
cd <racine_projet>\backend
.\venv\Scripts\Activate.ps1
python scripts\prepare_models.py
```

Le script télécharge automatiquement `intfloat/multilingual-e5-small` et l'enregistre dans `<racine_projet>/backend/models/multilingual-e5-small`.
Une fois le téléchargement terminé, l'application est configurée avec `TRANSFORMERS_OFFLINE=1` et `HF_HUB_OFFLINE=1` : **aucun accès Internet ne sera plus jamais sollicité**.

---

6. Étape 4 : Configuration des Variables d'Environnement (.env)

A. Les 3 Profils Matériels Prédéfinis

Pour adapter RailMind Lite aux capacités de votre serveur sans configuration manuelle fastidieuse, 3 profils sont préconfigurés dans `backend/env-profiles/` :

PROFIL	CIBLE MATÉRIELLE	RAM CONSEILLÉE	MODÈLE LLM	CONCURRENCE SIMULTANÉE	WORKERS BACKEND
Profil A	Machine bureautique / CPU modeste (Core i3/i5)	8 Go	`llama3.2:3b`	1 à 3 requêtes	1 worker
Profil B	Serveur d'entreprise CPU (Xeon / Core i7-i9, 8-16 cœurs)	16 à 32 Go	`llama3.2:3b`	5 à 15 requêtes	2 à 3 workers
Profil C	Serveur dédié avec carte graphique (GPU NVIDIA RTX/A-Series)	32 à 64 Go	`llama3.1:8b`	15 à 30 requêtes	4 workers

Activation du profil choisi :

Linux :

```
cd <racine_projet>
python backend/scripts/select_profile.py B  # Active le profil B (recommandé en production)
```

🪟 **Windows (PowerShell) :**

```
cd <racine_projet>
python backend\scripts\select_profile.py B
```

B. Dictionnaire des Variables Critiques (`backend/.env`)

Le fichier `backend/.env` contient la configuration du backend. Ajustez les variables suivantes selon votre infrastructure :

```
# ==========================================
# BASE DE DONNÉES POSTGRESQL
# ==========================================
# Format : postgresql://<utilisateur>:<mot_de_passe>@<hote>:<port>/<nom_base>
DATABASE_URL=postgresql://camrail_user:camrail_secure_password@localhost:5432/camrail_rda
DB_POOL_SIZE=10
DB_MAX_OVERFLOW=20
DB_POOL_TIMEOUT=30

# ==========================================
# SÉCURITÉ & AUTHENTIFICATION
# ==========================================
# Générez une clé secrète aléatoire : openssl rand -hex 32
SECRET_KEY=9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d1e0f
ACCESS_TOKEN_EXPIRE_MINUTES=480

# ==========================================
# MODÈLES IA LOCAUX (AIR-GAPPED)
# ==========================================
# Chemin relatif ou absolu vers le dossier des modèles d'embedding
MODELS_DIR=./models
EMBEDDING_MODEL=multilingual-e5-small

# Moteur LLM Ollama local (qwen2.5:3b recommandé pour CAMRAIL, ou llama3.2:3b)
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=qwen2.5:3b
OLLAMA_TIMEOUT_SECONDS=120
OLLAMA_NUM_CTX=4096
OLLAMA_NUM_PREDICT=300

# ==========================================
# RECHERCHE RAG & PAGINATION VIRTUELLE
# ==========================================
RAG_MIN_CONFIDENCE=0.18
CHUNK_SIZE_WORDS=150
CHUNK_OVERLAP_WORDS=30
DOCX_WORDS_PER_VIRTUAL_PAGE=450
EVIDENCE_MIN_SCORE=0.30

# ==========================================
# URLs PUBLIQUES & SÉCURITÉ CORS
# ==========================================
# Remplacer "localhost" par l'IP ou le nom DNS du serveur CAMRAIL
CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
FRONTEND_URL=http://localhost:3000
API_PUBLIC_URL=http://localhost:8000
```

C. Configuration du Frontend (`frontend/camrail-rda/.env.local`)

Dans `frontend/camrail-rda/.env.local` :

```
# URL de l'API Backend accessible depuis le navigateur client
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1

# Activer ou désactiver le bouton SSO Microsoft (true / false)
NEXT_PUBLIC_MICROSOFT_SSO_ENABLED=false
```

---

7. Étape 5 : Initialisation de la Base de Données (Alembic & Seed)

Une fois PostgreSQL configuré et accessible avec `DATABASE_URL` :

1. Appliquer les migrations de schéma et index vectoriels HNSW :

🐧 **Linux :**

```
cd <racine_projet>/backend
source venv/bin/activate
alembic upgrade head
```

🪟 **Windows :**

```
cd <racine_projet>\backend
.\venv\Scripts\Activate.ps1
alembic upgrade head
```

2. Initialisation des comptes utilisateurs :

🔒 **En Production (`APP_ENV=production`) :**

> ⚠️ **RÈGLE CRITIQUE DE SÉCURITÉ :** Les comptes de démonstration ne doivent **JAMAIS** être utilisés en production.
> - Le script `seed_demo.py` refuse catégoriquement de s'exécuter en production (`APP_ENV=production`).
> - Au démarrage, l'application vérifie si des comptes de démonstration par défaut existent et les désactive immédiatement (`demo_account_disabled`).
>
> Pour créer le premier compte administrateur en production avec saisie masquée (`getpass`) et validation de robustesse (>= 12 caractères, rejet des mots de passe triviaux) :

🐧 **Linux :**

```
python scripts/create_admin.py
```

🪟 **Windows :**

```
python scripts\create_admin.py
```

🧪 **En Développement Uniquement (`APP_ENV=development`) :**

Pour injecter le jeu documentaire factice et les comptes de test locaux :

🐧 **Linux :**

```
python seed_demo.py
```

🪟 **Windows :**

```
python seed_demo.py
```

---

8. Étape 6 : Démarrage du Backend FastAPI

Option 1 : Mode Développement Manuel (avec rechargement à chaud)

🐧 **Linux :**

```
cd <racine_projet>/backend
source venv/bin/activate
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

🪟 **Windows (PowerShell) :**

```
cd <racine_projet>\backend
.\venv\Scripts\Activate.ps1
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Option 2 : Mode Production Multi-Workers (Hautes Performances)

Ce mode lance plusieurs processus Uvicorn parallèles pour absorber la charge de centaines d'utilisateurs simultanés :

Linux :

```
cd <racine_projet>
chmod +x scripts/start_server.sh
./scripts/start_server.sh
```

Windows :

Double-cliquez sur `scripts\start_server.bat` ou lancez dans l'invite de commande :

```
cd <racine_projet>
scripts\start_server.bat
```

> **Validation :** Ouvrez votre navigateur sur **http://localhost:8000/docs** pour vérifier que l'API Swagger s'affiche.

---

9. Étape 7 : Installation & Démarrage du Frontend Next.js

A. Installation des dépendances Node.js

Linux :

```
cd <racine_projet>/frontend/camrail-rda
npm install
```

Windows (PowerShell) :

```
cd <racine_projet>\frontend\camrail-rda
npm install
```

B. Mode Développement

Linux & Windows :

```
npm run dev
```

L'application est immédiatement accessible sur **http://localhost:3000**.

C. Mode Production (Optimisé & Haute Vitesse)

En production, compilez les bundles statiques avec Turbopack pour des performances maximales :

Linux & Windows :

```
npm run build
npm run start -- -p 3000
```

---

10. Étape 8 : Déploiement Conteneurisé Clé en Main & Durcissement (Docker Compose)

RailMind Lite intègre une infrastructure conteneurisée hautement durcie respectant les standards de sécurité industrielle ferroviaire :

- **Images strictement épinglées** : Chaque image utilise une version immuable vérifiée (`pgvector:0.7.4-pg16` , `ollama:0.4.7` , `nginx:1.26.2-alpine` , `python:3.12.7-slim` , `node:20.18.0-alpine`).
- **Moindre privilège** : Conteneurs exécutés sous utilisateur non-root (`USER app` UID 1001), `cap_drop: ALL` , `no-new-privileges: true` , et système de fichiers racine en lecture seule (`read_only: true`).
- **Cloisonnement réseau** : Les ports de PostgreSQL (5432) et d'Ollama (11434) ne sont **plus publiés sur l'hôte**, limitant leur exposition au strict réseau virtuel interne Docker.
- **Rôle DB applicatif restreint** : L'application se connecte avec le rôle `railmind_app` (défini dans `deploy/postgres/init.sql`) sans aucun privilège de super-utilisateur.

A. Démarrage Standard (HTTP Interne)

```
cd <racine_projet>

# 1. Copier et adapter les variables d'environnement
cp env.example .env

# 2. Construire et lancer les services essentiels
docker compose up -d --build

# 3. Vérifier l'état de santé des services
docker compose ps
```

B. Déploiement Haute Sécurité HTTPS avec Reverse-Proxy Nginx

Le profil Compose `https` active un reverse-proxy Nginx frontal assurant :

- La terminaison TLS 1.2 / TLS 1.3 avec suites cryptographiques modernes et HSTS (`Strict-Transport-Security: max-age=31536000; includeSubDomains`).
- La redirection automatique de tout le trafic HTTP (port 80) vers HTTPS (port 443).
- Le routage de même origine (Frontend sur `/` , API sur `/api/`), éliminant tout besoin de CORS.
- La transmission en temps réel token par token du LLM (`proxy_buffering off` et `proxy_read_timeout 300s` sur la route de streaming).

1. Préparation des Certificats SSL/TLS

Option A : Environnement de Test ou Développement (Certificat Auto-Signé)

Générez un certificat auto-signé valide pour `localhost` et `railmind.camrail.net` :

- **Sous Linux (Bash) :**

  ```
  bash scripts/make_selfsigned_cert.sh
  ```

- **Sous Windows (PowerShell) :**

  ```
  powershell -ExecutionPolicy Bypass -File scripts\make_selfsigned_cert.ps1
  ```

Ces scripts génèrent `deploy/certs/privkey.pem` et `deploy/certs/fullchain.pem` (répertoire automatiquement exclu de git via `.gitignore`).

Option B : Production Ferroviaire CAMRAIL (Autorité de Certification Interne PKI / AD CS)

Pour déployer en production sans avertissement de sécurité sur les navigateurs des postes de travail :

1. Faites émettre un certificat serveur auprès de la **Direction des Systèmes d'Information (DSI) de CAMRAIL** ou de l'autorité racine Active Directory Certificate Services (AD CS).
2. Déposez la clé privée serveur au format PEM dans :
   `deploy/certs/privkey.pem` (définissez les droits stricts : `chmod 600`).
3. Concaténez le certificat du serveur, les certificats intermédiaires et le certificat racine CAMRAIL dans :
   `deploy/certs/fullchain.pem` .
4. Assurez-vous que l'autorité racine CAMRAIL est déployée via GPO dans le magasin "*Autorités de certification racines de confiance*" des postes clients.

2. Configuration du Frontend pour l'Origine Relative

En mode HTTPS frontal, configurez le frontend pour utiliser l'URL relative de même origine :

```
# Dans frontend/camrail-rda/.env.local (ou build arg Docker) :
NEXT_PUBLIC_API_URL=/api/v1
```

3. Lancement du Profil HTTPS

```
# Lancement combiné avec le profil Nginx
docker compose --profile https up -d --build
```

L'application est alors accessible de façon totalement sécurisée sur **https://railmind.camrail.net** ou **https://<ip_serveur>**.

C. Confidentialité & Gestion de la Dictée Vocale (Microphone)

Pour garantir une étanchéité absolue des données ferroviaires, **la dictée vocale dans l'assistant est DÉSACTIVÉE PAR DÉFAUT**.

> ⚠️ **Avertissement de Sécurité :** L'API `webkitSpeechRecognition` intégrée aux navigateurs (Google Chrome, Microsoft Edge) transmet les flux audio captés par le microphone vers les serveurs cloud de tiers (Google ou Microsoft) pour transcription vocale. Sur un réseau isolé ou pour des informations confidentielles d'exploitation, ce flux externe constitue un risque de fuite de données.

Si la politique de sécurité informatique de CAMRAIL autorise cette fonctionnalité sur votre poste :

1. Ouvrez `frontend/camrail-rda/.env.local`.
2. Définissez la variable suivante :

```
NEXT_PUBLIC_VOICE_INPUT_ENABLED=true
```

3. Redémarrez le frontend (`npm run dev` ou rebuild Docker). L'icône de microphone apparaîtra alors à droite de la barre de saisie.

---

11. Procédure de Transfert 100 % Hors-Ligne (Serveur Air-Gapped)

Dans le cas où le serveur final de CAMRAIL n'a **aucun accès Internet** :

Étape 1 : Sur une machine connectée (votre poste de travail)

1. **Cloner le projet** et installer les dépendances.
2. **Télécharger le modèle Ollama :**

```
ollama pull llama3.2:3b
```

3. **Archiver le dossier des modèles Ollama :**
   - *Linux :* `tar -czvf ollama_models.tar.gz -C ~/.ollama models`
   - *Windows (PowerShell) :* `Compress-Archive -Path "$env:USERPROFILE\.ollama\models" -DestinationPath "ollama_models.zip"`
4. **Préparer les modèles d'embedding :**

```
python backend/scripts/prepare_models.py
```

   (Cela peuple le dossier `backend/models/multilingual-e5-small`).
5. **Télécharger les packages Python (wheels) :**

```
pip download -r backend/requirements.txt -d ./python_wheels
```

6. **Copier l'ensemble** sur un support amovible sécurisé (clé USB ou disque externe).

Étape 2 : Sur le serveur isolé CAMRAIL

1. **Copier les fichiers du projet** à l'emplacement cible (ex: `/opt/camrail/railmind` ou `D:\camrail`).
2. **Restaurer les modèles Ollama :**
   - *Linux :* `sudo tar -xzvf ollama_models.tar.gz -C /usr/share/ollama/.ollama/`
   - *Windows :* Décompresser `ollama_models.zip` dans `C:\Users\<CompteSession>\.ollama\models`
3. **Installer les packages Python hors-ligne :**

```
cd backend
python -m venv venv
source venv/bin/activate  # ou .\venv\Scripts\Activate.ps1 sous Windows
pip install --no-index --find-links=../python_wheels -r requirements.txt
```

4. **Initialiser la base et lancer** les serveurs selon les sections 7, 8 et 9.

---

12. Guide de Dépannage & Erreurs Fréquentes

SYMPTÔME / MESSAGE D'ERREUR	CAUSE PROBABLE	SOLUTION IMMÉDIATE
`'uvicorn' n'est pas reconnu...`	L'environnement virtuel Python n'est pas activé dans le terminal.	Sous Windows : `.\venv\Scripts\Activate.ps1`. Sous Linux : `source venv/bin/activate`.
`'npm' n'est pas reconnu...`	Le dossier de Node.js n'est pas dans la variable `PATH` de Windows.	Vérifiez l'installation de Node.js ou lancez `node -v` pour diagnostiquer.
`HTTP 503 : Le modèle LLM local (Ollama) est indisponible`	Le service Ollama n'est pas démarré ou le modèle n'a pas été téléchargé.	Lancez Ollama (`ollama serve` ou démarrez l'application Windows) et vérifiez `ollama list`.
`fe_sendauth: no password supplied` ou erreur de connexion DB	Identifiants incorrects dans `DATABASE_URL` du fichier `.env`.	Vérifiez le mot de passe et le port (5432) dans `backend/.env`.
`extension "vector" is not available`	L'extension pgvector n'a pas été compilée ou activée dans PostgreSQL.	Exécutez `CREATE EXTENSION vector;` avec l'utilisateur postgres après avoir installé le paquet `postgresql-16-pgvector`.
`pdftoppm: command not found`	Poppler n'est pas installé ou absent du `PATH`.	Sous Linux : `sudo apt install poppler-utils`. Sous Windows : installez `poppler-windows` et ajoutez le dossier `bin` au `PATH`.
Avertissement d'infobulle de confiance insuffisante	L'assistant estime que les documents internes ne répondent pas avec certitude à la question.	Comportement normal de protection contre les hallucinations. Cliquez sur **Reformuler** ou enrichissez la base documentaire.

Document maintenu par l'équipe technique CAMRAIL — RailMind Lite v2.0.